

SSRGM: Software Strong Reliability Growth Model Based on Failure Loss

Huang Yafang

The Key Laboratory for Information
System Security
School of Software, Tsinghua
University
Beijing, China
huang_yafang65@yahoo.com.cn

Liu Yanzhao

China Information Technology
Security Evaluation Center
Beijing, China
cnitseclyz@163.com

Luo Ping

The Key Laboratory for Information
System Security
School of Software, Tsinghua
University
Beijing, China
luop@mail.tsinghua.edu.cn

Abstract—Failures that exist in software cause tremendous damage and bring large amount of loss. Thus the research of software trustworthiness has been attracted much attention from all over the world. By considering the impact of failure loss, the traditional definition of software reliability is redefined in this paper and improved to software strong reliability. Based on this new definition and traditional J-M model, a new model called SSRGM is proposed along with its mathematical relationship and implementation based on Markov Chain. Besides, through experiment and analysis, the conclusion that this model is correct and effective is drawn.

Keywords—software reliability; J-M model; failure loss; Markov chain

I. INTRODUCTION

With the rapid development of information technology, information infrastructure based on mass storage, telecommunications and high-speed computing ability plays an important role in many fields such as politics, economic, military, culture and so on. At the same time, many systems including but not limited to operation system, management information system, internet application program become larger and more complex, and losses caused by software errors increase rapidly and bring tremendous damage. Therefore, it is important to develop or choose the reliable software. Accordingly, software trustworthiness has attracted much attention from all over the world. Software reliability, as an important attribute of software trustworthiness, is studied by many research organizations.

Software reliability is defined as the probability of failure-free software operation for a specified period of time in a specified environment [1]. This definition is accepted by IEEE and ISO. There exist many models to measure, estimate and predict the software reliability based on software error data. A software reliability model describing such an error detection phenomenon is called a software reliability growth model (SRGM) [2].

There are various SRGMs and all these models can be classified to different categories. Fig.1 shows the classification of SRGMs by randomness.

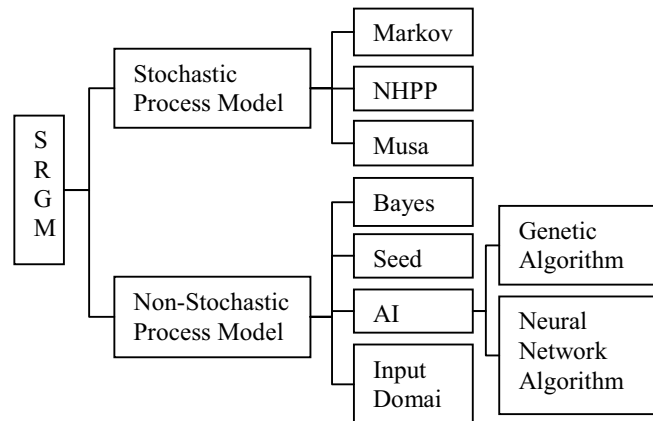


Figure 1. Classification of SRGM

The stochastic process model is an important method, for the reason that software failures which are subject to software errors occur at random time, that is, the software failure is a randomness problem. And it is a kind of practical method that possesses strong foundation of mathematics. J-M model [3], a basic SRGM based on the Markov Process, laid the foundation of the software reliability measurement. G-O model is a useful and classical SRGM described by nonhomogeneous Poisson process, which is basis on the assumption, that software failures accord with the Poisson distribution. Based on G-O model, Ref [4-9] provide many improvements and several optimized models. Shooman model [10] measures the software reliability with machine execution time and assumes that the failure time caused by software error obeys the exponential distribution. Musa model [11] combines the machine execution time and calendar time, along with the resource consumption during testing. Nelson model [12] considers the program input data as a data set which will be divided into input data subsets for normal operation and failure operation. While, S-W model [13] proposes that the failure detection rate increases along with the testing time, as well as the amount of remaining errors, and the failure time interval obeys Weibull distribution. Another model, such as Littlewood Bayesian model [14] uses Bayes method to improve J-M model and assumes that software errors

detected early in the testing process are different from errors detected later on.

Recently, some non-mathematical methods are proposed to measure software reliability, for example, artificial intelligence method. Ref [15] [16] employ Neural Network Model which is robust and whose implementation is comparative ripe, to measure and predict software reliability. Ref [17] proposes multivariate regression model by using genetic algorithm to solve the problem of local extremum in artificial intelligence method, but the implementation is complex.

II. SOFTWARE STRONG RELIABILITY

A. Software Reliability

In the traditional definition of software reliability, failure interval time is the basic measure criteria. The basic mathematical relationship is described as follows: let ξ be software failure time from the start running time, let $F_\xi(t)$ be the distribution function of ξ , let t be the arbitrary time, then $R_\xi(t)$ represents the reliability for time t .

$$R_\xi(t) = P_r\{\xi > t\} = 1 - F_\xi(t) \quad (1)$$

Failure rate $\lambda(t)$ is defined as the failure probability per unit time in the range $(t, t + \Delta t)$ under the condition that Δt is small enough.

$$\lambda(t) = \lim_{\Delta t \rightarrow 0} \frac{P_r\{t < \xi \leq t + \Delta t \mid \xi > t\}}{\Delta t} \quad (2)$$

Expansion of Formula (2):

$$\begin{aligned} \lambda(t) &= \lim_{\Delta t \rightarrow 0} \frac{P_r\{t < \xi \leq t + \Delta t \mid \xi > t\}}{\Delta t \cdot P_r\{\xi > t\}} \\ &= \lim_{\Delta t \rightarrow 0} \frac{F(t + \Delta t) - F(t)}{\Delta t \cdot R(t)} = \frac{f(t)}{R(t)} = \frac{-R'(t)}{R(t)} \end{aligned} \quad (3)$$

Where $f(t)$ is the density function of stochastic variable ξ . Assume that $R(0)=1$, because the software is considered completely reliable when initialized, and assume that failure rate $\lambda(t)$ is a constant, then we can get

$$R(t) = \exp\left\{-\int_0^t \lambda(s) ds\right\} = e^{-\lambda t} \quad (4)$$

According to its definition, software reliability represents the failure time, thus the expectation of software reliability can be defined as the mean lifetime of software, express as MTBF (Mean Time between Failures).

$$MTBF = \int_0^\infty t f(t) dt = \int_0^\infty \lambda t e^{-\lambda t} dt = \frac{1}{\lambda} \quad (5)$$

Besides, the traditional software reliability is measure by the software failure time. Specifically, time dimension include calendar time, clock time and machine execution time. When we discuss the problem of reliability, the unified time dimension needs to be chosen.

B. Software Strong Reliability

The measurement with failure time is proposed in the 70s. However, as errors which cause software failure become more various and more complex, we cannot easily only take failure time in consider. For the situation that 2 kinds of software have the same mean failure time, but one will cause larger loss when error occurs, we cannot consider that these two kinds of software have the same reliability. Therefore, an optimized definition of software reliability is requested.

To solve the problem that measurement by failure time cannot combine the actual effect brought by errors, we take the failure loss into consideration. Failure loss is the loss taken by the software failure, including the indirect economic loss, potential expense, restoring expense and so on. In the test stage, the failure loss is measured by restoring expense such as total LOC. This paper aims to the measurement and prediction model, the detail description and calculation process of failure loss will not be provided.

In this paper, by considering the failure loss, we use a new performance index called loss speed to measure software reliability.

Definition 1. Let T be the actual failure time, ω be the actual failure loss, for the current time t , loss speed is defined as follow:

$$v(t) = \frac{\omega}{T - t} \quad (6)$$

Loss speed describes the impact of the next failure on current. The more of the impact, the larger the loss speed is.

Compared with failure time which is the performance index of traditional SRGM, loss speed combines the impact of time and failure loss. As the run time goes on, the loss speed increases and the software trends to be more unreliable. The join of failure loss ω makes the impact of failure severity measurable. Two failures with the same failure time, the scale of 2 failures can be compared by their actual failure loss. So, the larger one is the one with larger loss and speeds up more quickly. When current time t is the initial run time of the software, loss speed v is exactly the average unit loss of a failure.

We redefine software reliability measured by failure loss speed to software strong reliability. Similarly, the model is defined as SSRGM: software strong reliability growth model. The basic mathematical relationship will be given in the following section.

C. Mathematical Relationship

Be similar with the definition of failure time, let δ be the software loss speed in a failure interval, and $F_\delta(v)$ represents the distribution of δ .

$$F_\delta(v) = P_r\{\delta \leq v\} \quad (7)$$

Definition 2. Software strong reliability $R_\delta(v)$ where v indicates the specified loss speed, is represented as follows:

$$R_\delta(v) = P_r\{\delta > v\} = 1 - F_\delta(v) \quad (8)$$

Formula above indicates that software strong reliability is the probability that software will not fail in the condition that the loss speed is v , that is the failure probability that actual loss speed is larger than the specific loss speed v .

Fig. 2 below describes the relationship between $F_\delta(v)$ and $R_\delta(v)$: along with the increasing of failure loss speed between failures, the reliability decreases and the failure probability increases accordingly. When the loss speed is 0, we can consider that the software will not fail and this software is completely reliable, so the reliability is 1. On the contrary, when $v \rightarrow +\infty$, the software tends to fail immediately, so it is less reliable and failure probability tends to be larger and approaches 1.

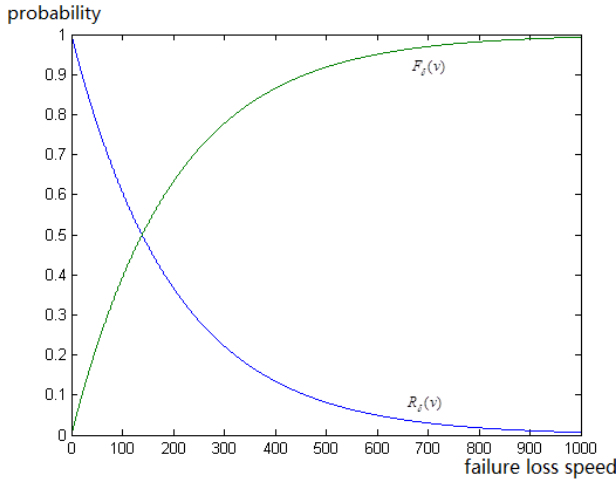


Figure 2. Relationship between $F_\delta(v)$ and $R_\delta(v)$

Be Similar with failure rate $\lambda(t)$, the new failure rate called strong failure rate is defined as follows:

Definition 3. Strong failure rate $\lambda(v)$ is the failure probability caused by increasing of unit loss speed, under the condition that the software works normally, when the loss speed is v and Δv is small enough.

$$\lambda(v) = \lim_{\Delta v \rightarrow 0} \frac{P_r\{v < \delta \leq v + \Delta v \mid \delta > v\}}{\Delta v} \quad (9)$$

Using the same computation process as we mentioned in the section A, we can get

$$R(v) = e^{-\lambda v} \quad (10)$$

MTBF is the basic performance index of software reliability in the traditional definition. Now since we changed the measurement and used the failure loss speed, we need change the basic index to a new one, which is MLSBF (mean loss speed between failures).

$$MLSBF = \int_0^\infty v f(v) dv = \int_0^\infty \lambda v e^{-\lambda v} dv = \frac{1}{\lambda} \quad (11)$$

MLSBF can be seen as the average unit failure loss for software. The larger its value is, the less reliable. It is better than the MTBF, because most of users focus on the average expense they will pay for the failure. Besides, when comparing

different software, time cannot measure severities of different failures in software.

D. Significance of MLSBF

- In measurement and comparison.

When measuring and comparing the reliability of different kinds of software, it is more practical using MLSBF than MTBF. For an organization, software is used to increase their working efficiency, decrease working cost and improve quality of working output as much as possible. Software is an important investment for them. And the failure loss speed is a comprehension performance index to the reliability and includes both the effect of failure time and actual average cost by software failure. The conclusion that software with higher MLSBF is less reliable than other software with lower MLSBF, poses practical significance. Then we can use MLSBF to choose software with lower expense.

- In prediction.

MLSBF can be used to predict the next unit failure loss caused by error, for example 500 dollars/day. It is useful, when the total budget is given, the decision when should stop using this software will be made since the unit failure loss can be predicted by using our model.

III. SSRGM

A. Model Implementation based on Markov Model

Basic Markov Reliable Model regards the occurrence of a failure as a stochastic process with sequential time and discrete status. Firstly, we will introduce the definition of Markov chain with sequential time and discrete status.

Definition 4. Stochastic process $X = \{X_t, t \in T\}$ is a set of stochastic variables defined in the probability space $(\Omega, \mathcal{F}, P)$, T is a sequential set (generally time) and ranges in $[0, +\infty)$, X_t is in the discrete status space S , and $S = \{s_1, s_2, s_3, \dots\}$. For arbitrary positive integer $n \geq 1$, nonnegative real number sequence $0 \leq t_1 < t_2 < \dots < t_n < t_{n+1}$ and arbitrary $i_1, i_2, \dots, i_n, i_{n+1} \in S$, if conditional probability meets the condition below:

$$\begin{aligned} P\{X_{t_{n+1}} = i_{n+1} \mid X_{t_1} = i_1, X_{t_2} = i_2, \dots, X_{t_n} = i_n\} \\ = P\{X_{t_{n+1}} = i_{n+1} \mid X_{t_n} = i_n\} \end{aligned} \quad (12)$$

Then we take $\{X_t, t \geq 0\}$ as sequential parameter Markov chain.

Based on this definition, we firstly define the mutex status of a software, i.e. $S = \{s_0, s_1\}$, s_0 represents the normal status while s_1 represents the failure status. Basic Markov reliable model assumes that each failure will be fixed immediately and will not trigger other failures. So the current status of software is only decided by the previous one status.

When $v = 0$, software is in the initial status s_0 . Besides, the last status or the balanced status is called the final status. Assume that the probability of a system which changes more than one status when loss speed changes Δv , is a higher order

infinitesimal, and negligible. That is to say, the status of system changes one time at most.

Based on the definition of strong failure rate $\lambda(v)$ described in Definition 3, software fails with the probability $\lambda\Delta v$ when failure loss speed changes Δv . Thus we can get that software will not fail with the probability $1-\lambda\Delta v$ accordingly. For the reason that software reliability is a matter of failure probability, we ignore the repairing of software failure status whether this failure can be repaired or not. So the status s_1 will not change to other status, we call this kind of status as absorbing state. Thus, the probability from status s_1 to s_0 is 0, and the probability from status s_1 to s_1 is 1. Fig. 3 shows the status transition diagram of software strong reliability.

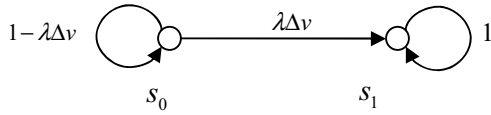


Figure 3. Markov transition diagram of software strong reliability

Let $P_0(v)$ be the probability of status s_0 when loss speed is v , and $P_1(v)$ stands for the probability of status s_1 . Accordingly, $P_0(v+\Delta v)$ represents the probability of status s_0 when loss speed is $v+\Delta v$, while $P_1(v+\Delta v)$ represents the probability of status s_1 . Based on the transition diagram showed in Fig.3, we can get the formula below:

$$P_0(v+\Delta v) = (1-\lambda\Delta v)P_0(v) + 0P_1(v) \quad (13)$$

Similarly, we can get:

$$P_1(v+\Delta v) = (\lambda\Delta v)P_0(v) + 1P_1(v) \quad (14)$$

We can get the simultaneous equation below:

$$\begin{cases} \frac{P_0(v+\Delta v) - P_0(v)}{\Delta v} = -\lambda P_0(v) \\ \frac{P_1(v+\Delta v) - P_1(v)}{\Delta v} = \lambda P_0(v) \end{cases} \quad (15)$$

When $\Delta v \rightarrow 0$, the left part of formula (15) become differential form and we can change it to:

$$\begin{cases} \frac{dP_0(v)}{dv} + \lambda P_0(v) = 0 \\ \frac{dP_1(v)}{dv} - \lambda P_0(v) = 0 \end{cases} \quad (16)$$

Associate with the initial condition of $P_0(0)=1$, solve this equation system and the solution is below:

$$\begin{cases} P_0(v) = e^{-\lambda v} \\ P_1(v) = 1 - e^{-\lambda v} \end{cases} \quad (17)$$

$P_1(v)$ is the probability that software fails when the failure loss speed is v , it can be seen as probability distribution function of loss speed.

$$F(v) = P_1(v) = 1 - e^{-\lambda v} \quad (18)$$

Accordingly, we can calculate the probability density function of loss speed:

$$f(v) = F'(v) = \lambda e^{-\lambda v} \quad (19)$$

Thus we can get the MLSBF:

$$MLSBF = \int_0^{\infty} v f(v) dv = \int_0^{\infty} \lambda v e^{-\lambda v} dv = \frac{1}{\lambda} \quad (20)$$

The solution we get by using Markov Chain is same with the result in formula (11). So our model is verified correct. The form of MLSBF is same with MTBF, but λ here is strong failure rate defined in Definition 3, while in MTBF λ is failure rate in traditional model.

B. J-M Assumption

Basic assumption is an important element for a SRGM, and it helps to simplify the computation process. It also lays the foundation of the model.

J-M model is one of the most important Markov reliability model [1]. The assumption J-M model proposed is described as follows:

- Testing environment is same with practical operation environment. So we can use the failure data collected in the testing stage to measure and predict the software reliability in practical use.
- The number of errors exists in software is a constant N_0 , but it is an unknown number.
- Each error in software is independent and triggers one failure each time. Meanwhile, each failure time interval is independent from one another.
- When detected, an error will be fixed immediately and the fixing time is negligible. An error will not trigger other errors.
- Failure rate is a constant in a failure interval, and is proportional to the remaining errors in software. In the i th failure time interval, failure rate function is:

$$\lambda(x_i) = \Phi(N_0 - i + 1) \quad (21)$$

Where Φ is the proportional constant, and x_i is the time variance in the i th failure time interval which starts from $(i-1)$ th failure.

C. New Assumption Considering Failure Loss

Because our SSRGM is based on J-M model, so based on these assumptions in above, SSRGM proposes similar assumptions as follows:

- Testing environment is same with practical operation environment. (Same with J-M)
- The total failure loss caused by failures in software is a constant W_0 , but it is an unknown number. (Developed)
- Each error in software is independent and triggers one failure each time. And the loss speed in each failure

time interval is independent from one another. (Developed)

- When detected, an error will be fixed immediately and the fixing time is negligible. An error will not trigger other errors. (Same with J-M)
- Strong failure rate is a constant in a failure interval, and is proportional to the remaining failure loss in software. In the i th failure time interval, the unit failure loss is v_i , strong failure rate function is:

$$\lambda(v_i) = \lambda(v(x_i)) = \Phi(W_0 - \sum_{k=1}^{i-1} w_k) \quad (22)$$

Where Φ is the proportional constant, and v_i is the unit failure loss or loss speed in the i th failure interval which starts from $(i-1)$ th failure. W_0 is the total failure loss and w_k is the failure loss of k th failure. (Developed)

D. Parameter Estimation Method

Except basic assumption, an appropriate parameter estimation method is also very important for a SRGM. And many developments focus on the parameter estimation method for a specific SRGM.

We choose the same method with J-M model which called Maximum Likelihood (ML) method to compute the value of Φ and W_0 , then we can further get the expression of strong failure rate $\lambda(v)$ and strong reliability $R(v)$.

Let $v_1, v_2, v_3, \dots, v_n$ be the sequence of failure loss speed or unit failure loss of different failure intervals in testing stage, we can get the expression below by using ML method:

$$L(W_0, \Phi) = \sum_{i=1}^n (\ln \Phi(W_0 - \sum_{k=1}^{i-1} w_k) - \Phi(W_0 - \sum_{k=1}^{i-1} w_k) v_i) \quad (23)$$

Let

$$\begin{aligned} \frac{\partial \ln(W_0, \Phi)}{\partial W_0} &= 0 \\ \frac{\partial \ln(W_0, \Phi)}{\partial \Phi} &= 0 \end{aligned} \quad (24)$$

We can get

$$\begin{cases} \sum_{i=1}^n \frac{1}{W_0 - \sum_{k=1}^{i-1} w_k} - \sum_{i=1}^n \Phi v_i = 0 \\ \frac{n}{\Phi} - \sum_{i=1}^n (W_0 - \sum_{k=1}^{i-1} w_k) v_i = 0 \end{cases} \quad (25)$$

Then we can get the results:

$$\begin{cases} \sum_{i=1}^n \frac{1}{W_0 - \sum_{k=1}^{i-1} w_k} = \frac{n \sum_{i=1}^n v_i}{W_0 \sum_{i=1}^n v_i - \sum_{i=1}^n (\sum_{k=1}^{i-1} w_k) v_i} \\ \Phi = \frac{n}{W_0 \sum_{i=1}^n v_i - \sum_{i=1}^n (\sum_{k=1}^{i-1} w_k) v_i} \end{cases} \quad (26)$$

Using error data collected from testing stage, approximate value of Φ and W_0 can be obtained. Next step, we can compute the average failure loss speed or unit failure loss according to formula (20) and (22):

$$MLSBF = \int_0^\infty v f(v) dv = \frac{1}{\lambda} = \frac{1}{\Phi(W_0 - \sum_{k=1}^{n-1} w_k)} \quad (27)$$

Then we can get the unit failure loss by formula above, this is used to predict the failure loss every unit time which generally measured by day.

IV. EXPERIMENT

A. Experiment Data

In experiment section, we use data of NTDS (Navy Tactical Data System) to test and verify SSRGM. It contains 31 failure records show in table I.

TABLE I. NTDS DATA

Number	Category	Actual failure time interval (day)
1	1	9
2	2	12
3	1	11
4	3	4
5	1	7
6	2	2
7	1	5
8	4	8
9	1	5
10	2	7
11	1	1
12	3	6
13	1	1
14	2	9
15	1	4
16	4	1
17	1	3
18	2	3
19	1	6
20	3	1
21	1	11
22	2	33
23	1	7
24	4	91
25	1	2
26	2	1
27	1	87
28	3	47
29	1	12

30	2	9
31	1	135

In order to compute loss speed, we need to get the loss caused by each failure. We use the catalog which indicates the failure severity as the failure loss and compute each loss speed. The loss speed in each failure interval is showed in Table II.

TABLE II. NTDS DATA WITH LOSS SPEED

Number	Loss Speed	Number	Loss Speed
1	0.111111111	17	0.333333333
2	0.166666667	18	0.666666667
3	0.090909091	19	0.166666667
4	0.75	20	3
5	0.142857143	21	0.090909091
6	1	22	0.060606061
7	0.2	23	0.142857143
8	0.5	24	0.043956044
9	0.2	25	0.5
10	0.285714286	26	2
11	1	27	0.011494253
12	0.5	28	0.063829787
13	1	29	0.083333333
14	0.222222222	30	0.222222222
15	0.25	31	0.007407407
16	4		

B. Result

To verify our model, the first 26 records are used to compute and predict total failure loss W_0 of NTDS and unit failure loss MLSBF for the 27th failure. The result we get is showed in Table III.

TABLE III. EXPERIMENT RESULT

Number	Name	Expression	Value
1	proportional constant	Φ	0.0297
2	total failure loss	W_0	76.5
3	unit failure loss	MLSBF	1.1814

C. Result Analysis

Previous SRGM focused on the failure time and improvements of traditional models. However SSRGM changes the view of software probability by combining failure loss and failure time. Because of the new index, it is hard to find specialized data with failure loss. In order to simplify the computation and solve the problem of source data, we consider the category as failure loss, because category presents the severity of each failure.

The actual total loss of 31 failures show in table I is 56. The total failure loss we can get by SSRGM is 76.5, but the actual amount of failure number is 34, then we can infer that the approximate value of W_0 is credible and our model is effective. In addition, MLSBF indicates that user needs to pay 1.1814 every day because of next software failure. This value can be also used to do comparison between different software.

From the result and analysis of experiment, we can get the conclusions below:

- **Correctness.** From one hand, verifiable mathematic model and computation process we mentioned in chapter II and III provide strong theoretical basis for our model. On the other hand, the value of total failure

loss W_0 that predicted in the experiment section approaches the actual value. So both from theory and practice, our model is verified to correct.

- **Effectiveness.** Traditional SRGMs pay attention to the failure time and regard it as the performance index of software reliability. Generally, failure time can't represent the severity of failure. And in practical use, users prefer using failure expenses to measure and predict software reliability by failure expenses, because occurrence of failures will cost much and produce loss. So it is practical and effective by combining the failure time and failure loss to measure reliability. In addition, our model can predict the total loss and the next unit failure loss, which are useful for users to make the budget and compare different software.

V. CONCLUSION

In this paper, we improve traditional software reliability to software strong reliability and propose a new model SSRGM based on J-M model. Combining the impact of failure loss and failure time, we can use SSRGM to predict the total expense caused by software failure and the average loss of unit time in a failure interval. By adding the impact of failure loss, our model adds the failure severity to the software reliability by total loss and MLSBF. Besides, through mathematical proof and data simulation, we can get the conclusion that our model is correct and effective.

SSRGM updates the software reliability from a perspective of failure loss, and it makes an important improvement which changes the performance index from time to loss speed. But the experimental data we can use is still for traditional model. So it is difficult to fully verify our model is effective. It remains future work to collect related data to verify and improve our model.

Although further optimization work is needed, we believe that our model will help to improve the work of the evaluation and prediction on software trustworthiness. It will contribute to make the software reliability model more effective and practical.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their detailed reviews and constructive comments, which have helped improve the quality of this paper. This work was supported by National Natural Science Foundation of China under Grant No. 90818021 and 9071803.

REFERENCES

- [1] IEEE 610.12:1990. Standard Glossary of Software Engineering Terminology. IEEE, 1990.
- [2] C.V.Ramamoothy and F.B.Bastani. Software reliability - status and perspectives. J.Syst. Software. vol 1, pp 315-318, 1980.
- [3] Jelinski Z, Moranda P. Software Reliability Research. In: W. Freiberger, Editor, Statistical Computer Performance Evaluation. New York: Academic Press, 1972: 465-484.
- [4] Yamada S, Ohba M, Osaki S. S-Shaped Reliability Growth Modeling for Software Error Detection. IEEE Transactions on Reliability, 1983, 32(5): 475-478.

- [5] Nara Takamasa, Nakata Masahiro, Ooshi Akihiro. Software Reliability Growth Analysis - Application of NHPP Models and its Evaluation. Proceedings of the International Symposium on Software Reliability Engineering, 1995: 250-255.
- [6] Pham H, Nordmann L, Zhang X. A General Imperfect Software Debugging Model with S-Shaped Fault-Detection Rate. IEEE Transactions on Reliability, 1999, 48(2): 169-175.
- [7] Chin-Yu Huang, Michael R. Lyu, Sy-Yen Kuo. A Unified Scheme of some Non-Homogenous Poisson Process Models for Software Reliability Estimation. IEEE Transactions on Software Engineering, 2003, 29(3): 261-269.
- [8] Zhao J, Liu H W, Cui G, et al. A Software Reliability Growth Model from Testing to Operation. Proceedings of IEEE International Conference on Software Maintenance, 2005: 691-694.
- [9] Zhao J, Liu H W, Cui G, et al. Software Reliability Growth Model with Change-Point and Environmental Function. Journal of Systems and Software, 2006, 79(11): 1578-1587.
- [10] Shooman M L. Probabilistic Models for Software Reliability Prediction. Proceedings of International Symposium on Fault-Tolerant Computing, 1972: 211-215.
- [11] Musa J D. A Theory of Software Reliability and its Application. IEEE Transactions on Software Engineering, 1975, 1(3): 312-327.
- [12] Nelson E. Estimating Software Reliability from Test Data. Microelectronics Reliability, 1977, 17(1): 67-73.
- [13] Shick G J, Wolverton R W. An Analysis of Competing Software Reliability Models. IEEE Transactions on Software Engineering, 1978, 4(2): 104-120.
- [14] Littlewood B. Stochastic Reliability Growth: A Model for Fault Removal in Computer Programs and Hardware Designs. IEEE Transactions on Reliability, 1981, 30(4): 313-320.
- [15] Karunanithi N, Malaiya Y K, Whitley D. Prediction of Software Reliability using Neural Networks. Proceedings of International Symposium on Software Reliability Engineering, 1991: 124-130.
- [16] Kanmani S, Rhymend V, Sankaranarayanan U V. Object-Oriented Software Fault Prediction using Neural Networks. Information and Software Technology, 2007, 49(5): 483-492.
- [17] Costa E O, Vergilio S R, Pozo A, et al. Modeling Software Reliability Growth with Genetic Programming. Proceedings of IEEE International Symposium on Software Reliability Engineering, 2005: 171-180.